Morning Cheat Sheet — read this and nothing else

Frontend Developer · WeMakeScholars · 07:20–08:00 · Learn nothing new today — revision only.

07:20 – 08:00. Nothing else. No new material.
If a line here doesn't ring a bell, open that file for 3 minutes and come back.

The 8 sentences that carry this interview

1. **Why Next.js:** *"Plain React ships an empty HTML shell that the browser fills in — bad for first paint and unreliable for crawlers. Next renders on the server, so the browser and Google both get real HTML immediately."*
2. **Rendering strategies:** *"SSG at build, SSR per request, ISR is SSG that regenerates on a timer, CSR in the browser. Bank listing pages → ISR. A student's own application page → SSR or CSR behind auth."*
3. **Server vs Client Components:** *"Server by default, ships zero JS; push* `'use client'` *down to the interactive leaves."*
4. **Core Web Vitals:** *"LCP under 2.5 s — is it there. CLS under 0.1 — does it stay still. INP under 200 ms — does it respond."*
5. **Closure:** *"A function that remembers the scope it was created in. It's how* `useState` *and debounce both work."*
6. **Keys:** *"Stable identity across renders. Index breaks the moment you delete or reorder — React reuses the wrong DOM node."*
7. **Four UI states:** *"Loading, error, empty, success. Empty is the one people forget."*
8. **Business tie-in (say once):** *"Since students find you through Google, server rendering and load speed here are lead generation, not polish."*

Your position — remember this

- **Not a fresher.** ~1 yr 4 mo production React + TypeScript at Standard Bank via Zensar. 10,000+ corporate users. Speak accordingly.
- **Immediate joiner.** Say it clearly — most candidates are on 60–90 days.
- **Target the top of the 4–6 band.** Earn it in the tech round, then say *"given the production experience, I'd be looking at the upper end."*
- **Lead with the performance story:** Lighthouse **62 → 88**, via code splitting (`React.lazy`) + **selector memoization** (`createSelector`). Measured first.
- **Your numbers:** 14-screen Business Card (60% faster processing) · 7-screen Recall Transactions · AOP audit, 50,000+/day · RabbitMQ notifications · QKart −40% re-renders.

The 4 questions your resume creates — have all 4 ready

1. **Gap since March?** Short, no apology, say what you did with the time. Then stop.
2. **Why leave Zensar?** Specialise in frontend · Hyderabad · product over services.
   Never criticise them.
3. **Java + Spring Boot, why frontend?** It's a strength — you can read an API and tell if the shape will hurt the UI.
4. **How much Next.js really?** "Contributed to the migration, not owned a Next app."
   Then immediately prove the reasoning with SSG/SSR/ISR.

TypeScript & Redux

- TS catches type errors **at build time**; types are **erased** at runtime, so it does **not** validate API responses.
- `interface` for object shapes/props · `type` for unions, tuples.
- `useState<User | null>(null)` and `useState<Loan[]>([])` — otherwise inferred as `null` and `never[]` .
- `unknown` = safe `any` ; forces a check before use.
- **RTK + Immer:** `state.items.push()` looks like mutation but produces new immutable state. That's the follow-up they ask.
- **Selector memoization:** a selector that `.filter()` s returns a **new array** time → new reference → re-render on every store update. `createSelector` fixes it. *(This is your dashboard story.)*
- **When NOT Redux:** local state → `useState` ; server data → React Query.
- **RTL principle:** test what the user sees, not internal state. `getByRole` first.

State management — server vs client ●

The distinction is the answer to almost every question here.

- **Server state** = a copy of data that lives on a server. Can go stale, is async, needs caching → **React Query** (or RTK Query).
- **Client state** = only exists in the browser. Never stale, synchronous → `useState` → Context → **Zustand** → **Redux Toolkit**.

The decision path: one component → `useState` · a few nearby → lift up · from an API → **React Query** · shared, rarely changes → Context · shared, changes often → **Zustand** · large app + team + complex flows → **Redux Toolkit**.

- **"Redux or React Query?"** is a trick question. Different problems, often both.

- **React Query gives you:** caching · deduplication · refetch on focus · retries · race conditions handled · loading and error states.
- `staleTime` = how long data counts as fresh (controls refetching).
  `gcTime` = how long unused data stays in memory (controls memory). Was `cacheTime` before v5.
- `invalidateQueries` after a mutation = "this list is now wrong, refetch it".
- **Zustand:** no Provider · store is a hook · `useStore(s => s.count)` subscribes to one value, so only that component re-renders · ~1 KB.
- **Context's weakness:** every consumer re-renders on any change, and it does no caching. Zustand fixes the first, React Query the second.
- **Your answer:** *"At Standard Bank we kept server data in Redux and hand-wrote loading flags and cache invalidation. I'd put that in React Query now and keep the store for real client state. On QKart, Context plus memoisation was the right size — Redux there would have been over-engineering."*

JavaScript

- `var` function-scoped · `let` / `const` block-scoped + TDZ · `const` ≠ immutable object
- Event loop: **microtasks (Promises) before macrotasks (setTimeout)** → `1 4 3 2`
- `Promise.all` parallel · `allSettled` all results · `race` first settled · `any` first success
- Arrow functions take `this` from the enclosing scope and can't be rebound
- `??` only falls back on null/undefined; `||` falls back on any falsy (`0` !)
- Spread = **shallow** copy · deep = `structuredClone`
- `fetch` **does not throw on 404/500** — check `res.ok`
- Debounce = after they stop (search) · Throttle = at most once per N ms (scroll)
- `typeof null === 'object'` · `NaN !== NaN` · `Array.isArray()`

React

- Re-render causes: own state · props · **parent re-rendered** · context value
- Reduce: `React.memo` → `useCallback` / `useMemo` → move state down → `children`
- Never mutate state — new reference or no re-render (`Object.is`)
- `setCount(c => c + 1)` when the new value depends on the old
- `useEffect` = sync with the outside world; cleanup `return`; don't use it for derived data
- `useMemo` value · `useCallback` function · `useRef` persists without re-rendering
- Hooks: top level only — React tracks them **by call order**
- Context re-renders every consumer; memoize the provider value
- `React.lazy` + `Suspense` = code splitting

Next.js

- `app/page.js` routing · `[id]` dynamic · `layout.js` · `loading.js` · `error.js`
- `fetch(url, { next: { revalidate: 60 } })` = ISR · `{ cache: 'no-store' }` = SSR
- `next/image` : WebP/AVIF, srcset, lazy, reserves space (CLS) · `priority` for the LCP image
- `metadata` / `generateMetadata` for per-page title & description
- Hydration = server HTML + client listeners; mismatches come from `Date` , `random` , `window`
- `<Link>` client-side nav + prefetch; `<a>` = full reload
- App Router → `next/navigation` ; Pages Router → `next/router`
- Only `NEXT_PUBLIC_*` reaches the browser — never a secret

CSS

- `box-sizing: border-box` · Flex = 1D, Grid = 2D
- `grid-template-columns: repeat(auto-fit, minmax(260px, 1fr))` = responsive, no media queries
- Mobile-first with `min-width` · viewport meta tag or nothing works
- `display:grid; place-items:center` to centre
- Animate `transform` / `opacity` only — they skip layout and paint

APIs

- 401 = who are you · 403 = I know, still no · 429 = rate limited
- `AbortController` in the effect cleanup → no stale state, no race condition
- CORS is a browser rule, fixed on the **backend**

System design — the 6 step framework

Clarify → Components → State → Data → Edge cases → Performance

- **Never start without asking 2 questions.** "Client side or API filtering?" "How many items?" "Does this need to rank on Google?"
- **Autocomplete:** debounce 400ms · four states · cancel the old request with `AbortController` · cache per query.
- **Listing page:** keep filters **in the URL** so links are shareable, back works, and the server can render it. Reset to page 1 when a filter changes.
- **Multi step form:** all data lives in the **parent**, steps are display only.
  Otherwise going back destroys what was typed.
- **Reusable component:** props say *what* not *how* · use `children` · spread `...rest` · a component with 15 boolean flags should be split.

Machine coding — the 6 things they're grading

1. `key={item.id}`, never the index
2. Derived data computed during render, not stored in state + effect
3. Empty state handled ("No results for…")
4. Loading **and** error states, ideally with a retry
5. Cleanup — `clearTimeout` / `clearInterval` / `abort`
6. **Talking while you type**

Opening & closing lines

"Tell me about yourself" → Present → Path → **Why here** (file 09 §1). Under 2 min.
Your closing question → *"What's the biggest frontend problem you're hoping this hire helps solve?"*

Filter questions — answer instantly, no hesitation:
18-month commitment ✓ · WFO ✓ · 1st & 3rd Saturdays ✓ · 10–11 AM login ✓
Salary → *"I saw the 4–6 LPA band and I'm comfortable in it; I'd expect to be placed based on how the technical rounds go."*
Never bid below 4.

The last 5 minutes before you walk in

- Close every tab except your GitHub and your two running projects.
- You know more than you think you do. Nobody expects perfection at this level — they expect someone who reasons clearly and doesn't bluff.
- **If you don't know something: say so, then reason out loud.** That answer has never lost anyone an offer. Bluffing has.
- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.